SECTION 4

7.a) Ram is developing a feature of an online apparel application as service. How should he handle the different requests to the service.

1.Establish a Centralized Request Portal

Create a self-service portal that allows users to easily submit and track service requests. This central hub should be intuitive and accessible, offering clear service descriptions and expected delivery timelines.

2. Define and Standardize Request Types

Develop a comprehensive service catalog that includes all available request types, along with required information and approval workflows. Standardizing request types ensures consistency, reduces confusion, and facilitates automation.

3. Automate Repetitive Requests

Utilize ITSM tools to automate common service requests, such as password resets or software installations. Automation reduces manual workload, decreases response times, and minimizes errors.

4. Prioritize Requests Based on Impact and Urgency

Implement a prioritization model to categorize requests by their urgency and business impact. This ensures that high-priority requests are addressed promptly without delaying routine requests.

5. Set Clear SLAs and Monitor Performance

Define measurable Service Level Agreements (SLAs) for each request type and use Key Performance Indicators (KPIs) to monitor compliance. Regular tracking helps identify bottlenecks and areas for improvement.

6. Enable Knowledge Base Integration

Integrate a knowledge base into your self-service portal to provide users with FAQs, how-to guides, and troubleshooting tips. Empowering users to resolve minor issues independently reduces the request load on IT teams.

7. Implement Approval Workflows

Ensure that service requests follow a structured approval workflow, especially for access and hardware-related requests. This prevents unauthorized actions and supports compliance requirements.

8. Categorize and Tag Requests for Reporting

Use metadata, categories, and tags to organize and classify service requests. This improves reporting accuracy and enables better data analytics and trend identification.

9. Train and Empower IT Support Staff

Provide continuous training to support staff to ensure they are equipped to handle a variety of service requests efficiently. Well-trained teams resolve requests faster and improve customer satisfaction.

10. Review and Improve Request Fulfillment Processes

Regularly assess the request management process to identify gaps, improve workflows, and implement best practices. Collect feedback from end-users and support teams to guide enhancements.

7.b) Users of Instagram, a photo sharing application, can share photographs not only with Instagram friends but also with friends on other social networking applications such as Twitter and Facebook. Explain how is this possible.

. Linking Social Accounts for Cross-Posting

Instagram provides an option where users can connect their Instagram profile with other social media accounts, such as Facebook and Twitter. This linking creates a seamless bridge, allowing posts made on Instagram to be automatically shared on these platforms without additional steps. When setting up their profile or through settings, users can authorize Instagram to access and post on their behalf to these connected accounts.

. Sharing Photos on Multiple Platforms

When a user uploads a photo on Instagram, after editing and adding caption details, the app displays sharing options that include toggles for linked social networks. By simply toggling these options on, the user authorizes Instagram to simultaneously post that photo to their Facebook or Twitter accounts. This process maximizes the photo's reach by allowing the content to appear in multiple social feeds at once.

. Benefits of Cross-Platform Sharing

- Increased Reach & Engagement: Sharing photos across networks like Facebook and Twitter amplifies audience size beyond Instagram followers.
- Convenience: Users don't need to manually upload the same photo to several apps.
- Consistency: Maintains a unified online presence across platforms easily.

. Technical Underpinnings

Instagram leverages API integrations with platforms such as Facebook and Twitter. These APIs allow Instagram to authenticate the user on different platforms and perform authorized actions like posting images or videos. The integration ensures smooth and immediate content sharing while respecting privacy and permissions granted by the user.

8.a) **Develop the data access layer of the Employee Management Application to perform the database operations given below using Spring Data JPA**
Add the operation given below using Spring Data JPA: Update the employee Contact Number for the given employee id.

Create a JPA entity class representing the Employee with fields like id and employe Contact Number.

```
import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;
```

```
@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String employeeContactNumber;
    // Other employee fields like name, etc.
    // Getters and Setters
    public Long getId() {
        return id;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public String getEmployeeContactNumber() {
        return employeeContactNumber;
    }
    public void setEmployeeContactNumber(String employeeContactNumber) {
        this.employeeContactNumber = employeeContactNumber;
    }
}
```

Create the Employee Repository Interface.

Extend JpaRepository to leverage Spring Data JPA's built-in CRUD operations. You can also define a custom query method for updating the contact number.

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import jakarta.transaction.Transactional;
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    @Modifying
    @Transactional
```

```
    @Query("UPDATE Employee e SET e.employeeContactNumber = :contactNumber WHERE e.id
= :employeeId")
    int updateEmployeeContactNumber(@Param("employeeId") Long employeeId,
@Param("contactNumber") String contactNumber);
}
```

- @Modifying: Indicates that this query modifies the database.
- @Transactional: Ensures the update operation is executed within a transaction.
- @Query: Defines the JPQL query to update the employeeContactNumber.
- @Param: Binds method parameters to named parameters in the query.

A service layer encapsulates business logic and interacts with the repository.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;
    public boolean updateEmployeeContactNumber(Long employeeId, String newContactNumber) {
        int updatedRows = employeeRepository.updateEmployeeContactNumber(employeeId,
newContactNumber);
        return updatedRows > 0;
    }
}
```

- **Expose the Operation via a Controller (for Web Applications):**
- A REST controller can expose an endpoint to trigger the update.

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class EmployeeController {
    @Autowired
    private EmployeeService employeeService;
```

```
    @PutMapping("/employees/{id}/contact")
    public ResponseEntity<String> updateEmployeeContact(@PathVariable Long id, @RequestParam String newContactNumber) {
        boolean updated = employeeService.updateEmployeeContactNumber(id, newContactNumber);
        if (updated) {
            return ResponseEntity.ok("Employee contact number updated successfully.");
        } else {
            return ResponseEntity.notFound().build();
        }
    }
}
```

This setup provides a complete data access layer for updating the employee contact number using Spring Data JPA.

8.b) Create a REST controller class to perform CRUD operations on product and corresponding request and response DTOs.The product class should contain three data members product name, product category, price. User proper SpringBoot annotations.

A REST controller class using Spring Boot to perform CRUD operations on a Product entity. Ill al provide the corresponding request and response DTOs (Data Transfer Objects) to handle the data sent and received from the API.

1. Introduction

Spring Boot provides a powerful framework for building RESTful web services in Java, using annotations and auto-configuration to simplify development. This report demonstrates how to create a REST controller to perform CRUD (Create, Read, Update, Delete) operations on a Product entity, using DTOs (Data Transfer Objects) for request and response handling.

2. Product Entity Definition

The Product entity contains the following data members:

Field Name	Data Type	Description
productName	String	Name of the product
productCategory	String	Category of the product
price	Double	Price of the product

3. Project Structure

```
com.example.productapp
├── controller
│   └── ProductController.java
├── dto
│   ├── ProductRequestDTO.java
│   └── ProductResponseDTO.java
```

```
├── model
│   └── Product.java
├── service
│   └── ProductService.java
├── repository
│   └── ProductRepository.java
├── ProductAppApplication.java
```

4. Implementation

+

a) Product Model Class

```
package com.example.productapp.model;

public class Product {
    private Long id;
    private String productName;
    private String productCategory;
    private Double price;

    // Constructors, Getters, Setters
}
```

b) DTO Classes

ProductRequestDTO.java

```
package com.example.productapp.dto;

public class ProductRequestDTO {
    private String productName;
    private String productCategory;
    private Double price;

    // Constructors, Getters, Setters
}
```

ProductResponseDTO.java

```
package com.example.productapp.dto;

public class ProductResponseDTO {
    private Long id;
    private String productName;
    private String productCategory;
    private Double price;

    // Constructors, Getters, Setters
}
```

c) Product Repository

For simplicity, use an in-memory list instead of a real database.

```
package com.example.productapp.repository;

import com.example.productapp.model.Product;
import org.springframework.stereotype.Repository;
```

```
import java.util.*;

@Repository
public class ProductRepository {
    private Map<Long, Product> productData = new HashMap<>();
    private Long idCounter = 1L;

    public Product save(Product product) {
        product.setId(idCounter++);
        productData.put(product.getId(), product);
        return product;
    }

    public Optional<Product> findById(Long id) {
        return Optional.ofNullable(productData.get(id));
    }

    public List<Product> findAll() {
        return new ArrayList<>(productData.values());
    }

    public Product update(Product product) {
        productData.put(product.getId(), product);
        return product;
    }

    public void deleteById(Long id) {
        productData.remove(id);
    }
}
```

d) Product Service

```
package com.example.productapp.service;

import com.example.productapp.dto.ProductRequestDTO;
import com.example.productapp.model.Product;
import com.example.productapp.repository.ProductRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public Product createProduct(ProductRequestDTO request) {
        Product product = new Product();
        product.setProductName(request.getProductName());
```

```
        product.setProductCategory(request.getProductCategory());
        product.setPrice(request.getPrice());
        return repository.save(product);
    }

    public Optional<Product> getProductById(Long id) {
        return repository.findById(id);
    }

    public List<Product> getAllProducts() {
        return repository.findAll();
    }

    public Product updateProduct(Long id, ProductRequestDTO request) {
        Product product = repository.findById(id).orElseThrow(() -> new RuntimeException("Product not
found"));
        product.setProductName(request.getProductName());
        product.setProductCategory(request.getProductCategory());
        product.setPrice(request.getPrice());
        return repository.update(product);
    }

    public void deleteProduct(Long id) {
        repository.deleteById(id);
    }
}
```

e) Product REST Controller

```
package com.example.productapp.controller;

import com.example.productapp.dto.ProductRequestDTO;
import com.example.productapp.dto.ProductResponseDTO;
import com.example.productapp.model.Product;
import com.example.productapp.service.ProductService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<ProductResponseDTO> createProduct(@RequestBody ProductRequestDTO
request) {
        Product product = service.createProduct(request);
        ProductResponseDTO response = mapToResponseDTO(product);
        return ResponseEntity.ok(response);
```

```
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponseDTO> getProduct(@PathVariable Long id) {
        return service.getProductById(id)
                .map(product -> ResponseEntity.ok(mapToResponseDTO(product)))
                .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping
    public ResponseEntity<List<ProductResponseDTO>> getAllProducts() {
        List<ProductResponseDTO> products = service.getAllProducts().stream()
                .map(this::mapToResponseDTO)
                .collect(Collectors.toList());
        return ResponseEntity.ok(products);
    }

    @PutMapping("/{id}")
    public ResponseEntity<ProductResponseDTO> updateProduct(@PathVariable Long id,
@RequestBody ProductRequestDTO request) {
        Product product = service.updateProduct(id, request);
        return ResponseEntity.ok(mapToResponseDTO(product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        service.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }

    private ProductResponseDTO mapToResponseDTO(Product product) {
        return new ProductResponseDTO(
                product.getId(),
                product.getProductName(),
                product.getProductCategory(),
                product.getPrice()
        );
    }
}
```

5. Conclusion

This report demonstrated the creation of a complete Spring Boot application for CRUD operations on a Product entity using RESTful APIs. Key highlights:

Separation of concerns using DTOs, services, and repositories.

Proper use of Spring annotations:

@RestController

@RequestMapping, @PostMapping, @GetMapping, @PutMapping, @DeleteMapping

@RequestBody and @PathVariable

Simplified in-memory storage for demonstration purposes.